Evolv - Documento Inicial do Projeto

1. Objetivo

O Evolv será um aplicativo mobile para planejamento e acompanhamento de treino e alimentação. O objetivo é criar um plano de longo prazo para o usuário, entregar a rotina semanal de forma automática e realizar ajustes pontuais conforme feedback, progresso e mudanças na rotina.

O app não deve trocar treino ou dieta sem motivo. A proposta é manter consistência, acompanhar evolução e ajustar apenas quando houver sinais relevantes, como falta de progresso, dificuldade de aderência, mudança de horário, indisponibilidade para treinar, queda de motivação ou restrições informadas pelo usuário.

2. Proposta do produto

O aplicativo funcionará como um assistente pessoal de treino e dieta com suporte de inteligência artificial. A IA será usada para conversar com o usuário, interpretar pedidos e explicar decisões, enquanto as regras principais do plano serão controladas pelo próprio sistema.

Principais recursos previstos:

- criação de plano inicial de treino e alimentação;
- entrega semanal dos treinos e refeições;
- notificações de treino, refeições e check-ins;
- calendário de treino com possibilidade de reorganização;
- chat específico para treino;
- chat específico para dieta;
- registro opcional de carga, medidas, fotos e alimentação;
- ajustes graduais com base no histórico do usuário.

3. Funcionamento

O usuário fará um onboarding inicial com dados pessoais, objetivo, rotina, nível de experiência, disponibilidade semanal, preferências e restrições.

A partir disso, o app criará um plano base. Esse plano será pensado para médio e longo prazo, mas apresentado ao usuário em formato semanal.

Exemplo de uso:

- o app informa que hoje é dia de costas;
- o usuário diz que não poderá treinar hoje;
- a IA entende o pedido;
- o sistema move o treino para outro dia;
- o restante da semana é reorganizado sem alterar todo o plano.

4. Estrutura do aplicativo

O app terá navegação por abas:

- Hoje: treino do dia, refeições do dia, lembretes e check-in rápido.
- Treino: calendário, treino atual, histórico, cargas opcionais e IA de treino.

- Dieta: plano alimentar, substituições, refeições e IA de dieta.
- Progresso: peso, medidas, fotos opcionais e evolução.
- Conta: dados do usuário, objetivo, preferências, restrições e notificações.

5. Arquitetura da solução

A solução será dividida em três partes principais:

- Aplicativo mobile: interface usada pelo usuário.
- Backend: API, regras de negócio, autenticação, planner e integrações.
- Banco de dados: armazenamento de perfil, planos, histórico e progresso.

A inteligência artificial não deve alterar o banco de dados livremente. Ela interpreta a conversa e aciona funções internas do sistema. As mudanças reais no plano passam por regras definidas no backend.

6. Planner

O Planner será um módulo interno do backend. Ele não é uma IA separada.

Responsabilidades do Planner:

- montar a semana de treino;
- organizar refeições;
- mover treinos no calendário;
- manter o plano estável quando não houver motivo para mudança;
- aplicar progressão básica de carga;
- sugerir ajustes pequenos quando houver feedback recorrente;
- preservar a lógica do plano de longo prazo.

Resumo:

- IA: entende e conversa.
- Planner: aplica regras e organiza o plano.
- Banco de dados: guarda histórico e informações do usuário.

7. Uso de IA

No início, não será treinado um modelo próprio. A recomendação é usar OpenAI API ou Gemini API.

Motivos:

- menor custo inicial;
- desenvolvimento mais rápido;
- não exige grande volume de dados;
- permite validar o produto antes de investir em modelo próprio.

A IA será usada para:

- interpretar mensagens;
- responder dúvidas;
- explicar ajustes;

- gerar sugestões iniciais;
- transformar pedidos em ações internas do sistema.

8. MVP

A primeira versão deve validar o funcionamento principal do produto.

Funcionalidades do MVP:

- cadastro;
- onboarding curto;
- plano inicial de treino;
- plano alimentar simples;
- tela Hoje;
- calendário de treino;
- chat de treino;
- chat de dieta;
- check-in rápido;
- notificações;
- ajuste básico de agenda.

Ficam para versões futuras:

- fotos de progresso;
- análise de imagem;
- integração com Google Calendar e Apple Calendar;
- scanner de alimentos;
- wearables;
- gráficos avançados;
- lista de compras;
- modelo próprio de IA.

9. Stack recomendada

Recomendação inicial:

- Mobile: React Native com Expo.
- Backend: Node.js com TypeScript, usando NestJS ou Fastify.
- Banco de dados: Supabase com PostgreSQL.
- IA: OpenAI API ou Gemini API.
- Notificações: Expo Notifications.

Essa stack é adequada para aprendizado, portfólio, GitHub e futura publicação do app.

10. Etapas de desenvolvimento

Ordem recomendada:

1. Definir escopo final do MVP.

2. Desenhar as telas principais.

3. Modelar o banco de dados.

4. Criar o app base com navegação por abas.

5. Criar o backend.

6. Implementar o Planner inicial sem IA.

7. Integrar a API de IA.

8. Adicionar notificações.

9. Refinar experiência e interface.

10. Preparar README, prints, vídeo curto e publicação no LinkedIn.

11. Aprendizado e portfólio

O projeto é relevante para aprendizado porque envolve produto, interface, backend, banco de dados, autenticação, IA, notificações, regras de negócio e documentação.

No GitHub e LinkedIn, o projeto pode ser apresentado como:

"Aplicativo mobile com IA para planejamento de treino e dieta, usando React Native, Node.js, Supabase e API de LLM. O sistema cria planos de longo prazo, entrega rotinas semanais e ajusta treino/dieta com base em feedback do usuário."

12. Próximo passo

Antes de iniciar o código, é necessário fechar:

- nome definitivo;
- escopo final do MVP;
- estrutura do onboarding;
- primeira versão do plano de treino;
- primeira versão do plano alimentar;
- escolha entre OpenAI API e Gemini API.

Recomendação: iniciar com app mobile, onboarding curto, treino e dieta simples, Planner com regras básicas e integração com IA apenas depois da estrutura principal estar funcionando.